Command Line Interface Cheat Sheet

Windows CMD · PowerShell · Bash (Linux and macOS) · Git · Docker

How to read this sheet. Each row shows the same task in all three shells. Commands are in monospace; *grey italics* explain flags or name an alias for the same command. 🔒 needs administrator, root or sudo. ⚠ destructive; check before running. (Linux) or (macOS) marks a command that exists only on that platform. ★ marks the everyday essentials. Click any command to copy it.

Everyday Essentials

The commands most people use every day, in short form. Click a task name for the full entry with more options and notes.

Task	CMD	PowerShell	Bash
★ **Change directory**	`cd C:\Users` `cd ..\Desktop`	`cd "C:\Program Files"` `Set-Location ~\Desktop`	`cd /home/user/docs` `cd ~/Desktop`
★ **List files**	`dir` `dir /A` *(include hidden)*	`Get-ChildItem` *(alias: ls, dir, gci)* `ls -Force` *(include hidden)*	`ls` `ls -la` *(long, incl. hidden)*
★ **Copy files**	`copy file.txt C:\Backup` `copy *.txt D:\Backup`	`Copy-Item file.txt ~\Backup` *(alias: cp)* `Copy-Item *.txt -Destination C:\Backup`	`cp file.txt ~/Backup` `cp -i file.txt ~/Backup` *(prompt before overwrite)*
★ **Move or rename**	`move file.txt C:\New` `ren old.txt new.txt`	`Move-Item file.txt C:\New` *(alias: mv)* `Rename-Item old.txt new.txt`	`mv file.txt ~/new/` `mv old.txt new.txt`
★ **Delete files**	`del oldfile.txt` `del *.tmp`	`Remove-Item oldfile.txt` *(alias: rm, del)* `Remove-Item *.tmp -WhatIf` *(dry run)*	`rm oldfile.txt` `rm -i file.txt` *(confirm)*
★ **View file contents**	`type readme.txt`	`Get-Content readme.txt` *(alias: cat, type, gc)*	`cat readme.txt` `less readme.txt` *(paginated; q to quit)*
★ **Find files**	`dir /S /B *.txt` `where /R C:\Projects *.log`	`Get-ChildItem -Recurse -Filter *.txt` `gci -r -Include *.log,*.txt`	`find . -name "*.txt"` `find /home -type f -name "*.log"`
★ **Search text in files**	`findstr "error" log.txt` `findstr /S /I "text" *.*` *(recursive, ignore case)*	`Select-String -Path log.txt -Pattern error` *(alias: sls)* `sls error *.log -CaseSensitive`	`grep "error" log.txt` `grep -ri "error" /var/log/` *(recursive, ignore case)*
★ **List processes**	`tasklist` `tasklist /V` *(verbose)*	`Get-Process` *(alias: ps, gps)* `Get-Process -Name chrome`	`ps aux` `ps aux | grep nginx`
★ **Kill process**	`taskkill /IM notepad.exe` `taskkill /PID 1234 /F` *(force)*	`Stop-Process -Name notepad` *(alias: kill)* `Stop-Process -Id 1234 -Force`	`kill 1234` *(SIGTERM, polite)* `kill -9 1234` *(SIGKILL, force)*

Task	CMD	PowerShell	Bash
★ **Ping host**	`ping google.com` *(4 packets)* `ping -n 10 192.168.1.1`	`Test-Connection google.com` `Test-Connection 8.8.8.8 -Count 5`	`ping google.com` *(until Ctrl+C)* `ping -c 5 google.com`
★ **IP configuration**	`ipconfig` `ipconfig /all` *(MAC, DNS, DHCP)*	`Get-NetIPConfiguration` `Get-NetIPAddress -AddressFamily IPv4`	`ip addr` *(Linux; short form: ip a)* `ifconfig` *(macOS; deprecated on Linux)*
★ **Show environment variables**	`set` `set PATH` *(all starting with PATH)*	`Get-ChildItem Env:` *(or: dir env:)* `$env:PATH`	`printenv` `env`
★ **Command history**	`doskey /history` `F7` *(popup list)*	`Get-History` *(alias: h, history)* `Get-Content (Get-PSReadLineOption).HistorySavePath` *(all sessions)*	`history` `history 20` *(last 20)*
★ **Get help**	`dir /?` `help copy`	`Get-Help Get-Process -Examples` `Get-Help *-Service` *(search by name)*	`man ls` `ls --help`

File and Directory Management

Task	CMD	PowerShell	Bash	Notes

★ **Change directory**	`cd C:\Users`
	`cd ..\Desktop`
	`cd /d D:\Data` *(switch drive)*
	`D:` *(switch drive only)*
		`cd "C:\Program Files"`
		`Set-Location ~\Desktop`
		`cd -` *(previous dir, PS 6+)*
			`cd /home/user/docs`
			`cd ~/Desktop`
			`cd -` *(previous dir)*
			`cd` *(home)*
				cd .. goes to the parent. ~ is the home directory in PowerShell and Bash. CMD needs /d to change drive and directory at once.

★ **List files**	`dir`
	`dir /A` *(include hidden)*
	`dir /S` *(recursive)*
	`dir /B` *(bare names)*
	`dir /O-D` *(newest first)*
		`Get-ChildItem` *(alias: ls, dir, gci)*
		`ls -Force` *(include hidden)*
		`Get-ChildItem -Recurse`
		`ls *.txt`
			`ls`
			`ls -la` *(long, incl. hidden)*
			`ls -lh` *(readable sizes)*
			`ls -lt` *(newest first)*
			`ls -R` *(recursive)*	

Show current directory	`cd` *(no arguments)* `echo %CD%`	`Get-Location` *(alias: pwd)* `$PWD`	`pwd`	
Create directory	`mkdir MyReports` `md Folder1\Folder2` *(nested is fine)*	`mkdir ~/Projects/New` `New-Item -ItemType Directory MyReports`	`mkdir MyReports` `mkdir -p a/b/c` *(nested)*	
Remove directory	`rmdir OldFolder` *(empty only)* `rmdir /S /Q OldFolder` *(with contents, no prompt)*	`Remove-Item OldFolder -Recurse -Force` `Remove-Item OldFolder -Recurse -WhatIf` *(preview)*	`rmdir OldFolder` *(empty only)* `rm -r OldFolder` `rm -rf OldFolder` *(no prompts)*	⚠ Recursive deletes bypass the Recycle Bin / Trash. There is no undo.
Directory tree	`tree C:\Projects` `tree /F` *(include files)*	`tree /F` *(same tree.com as CMD)* `Get-ChildItem -Recurse -Name`	`tree ~/Projects` `tree -L 2` *(limit depth)*	tree is not preinstalled on most Linux distros or macOS: apt install tree, brew install tree.

Task	CMD	PowerShell	Bash	Notes
★ **Copy files**	`copy file.txt C:\Backup` `copy *.txt D:\Backup`	`Copy-Item file.txt ~\Backup` *(alias: cp)* `Copy-Item *.txt -Destination C:\Backup`	`cp file.txt ~/Backup` `cp -i file.txt ~/Backup` *(prompt before overwrite)*	

Copy directories	`xcopy C:\Src D:\Dst /E /I`
	`robocopy C:\Src D:\Dst /E`
	`robocopy C:\Src D:\Dst /MIR` *(mirror; deletes extras)*
		`Copy-Item folder -Recurse -Destination dest`	`cp -r folder/ ~/dest`
			`cp -a folder/ ~/dest` *(preserve attributes)*
			`rsync -av src/ dest/`
				robocopy and rsync are the robust choices for large copies: both can resume and mirror. ⚠ /MIR and rsync --delete remove files at the destination.

★ **Move or rename**	`move file.txt C:\New` `ren old.txt new.txt`	`Move-Item file.txt C:\New` *(alias: mv)* `Rename-Item old.txt new.txt`	`mv file.txt ~/new/` `mv old.txt new.txt`	

★ **Delete files**	`del oldfile.txt`
	`del *.tmp`
	`del /P file.txt` *(prompt)*
		`Remove-Item oldfile.txt` *(alias: rm, del)*
		`Remove-Item *.tmp -WhatIf` *(dry run)*
			`rm oldfile.txt`
			`rm -i file.txt` *(confirm)*
			`rm *.tmp`
				⚠ No Recycle Bin. In PowerShell, -WhatIf previews any destructive cmdlet.

★ **View file contents**	`type readme.txt`	`Get-Content readme.txt` *(alias: cat, type, gc)*	`cat readme.txt` `less readme.txt` *(paginated; q to quit)*	
Create empty file	`type nul > notes.txt` `copy con notes.txt` *(type text, then Ctrl+Z, Enter)*	`New-Item -ItemType File notes.txt` *(alias: ni)*	`touch notes.txt` `> notes.txt`	touch also updates the timestamp of an existing file. New-Item errors if the file exists unless you add -Force.

★ **Find files**	`dir /S /B *.txt`
	`where /R C:\Projects *.log`
		`Get-ChildItem -Recurse -Filter *.txt`
		`gci -r -Include *.log,*.txt`
		`gci -r | Where LastWriteTime -gt (Get-Date).AddDays(-1)`
			`find . -name "*.txt"`
			`find /home -type f -name "*.log"`
			`find . -mtime -1` *(modified in last day)*
			`find . -size +100M`	

Which command runs	`where python`	`Get-Command python`
		`Get-Command ls` *(shows what an alias points to)*
			`which python`
			`type ls` *(shows aliases and builtins too)*
			`command -v python`	

Compare files	`fc file1.txt file2.txt` `fc /B a.bin b.bin` *(binary)*	`Compare-Object (Get-Content a.txt) (Get-Content b.txt)` *(alias: diff)*	`diff file1.txt file2.txt` `diff -u a.txt b.txt` *(unified format)*	PowerShell's diff alias is not Unix diff: it compares sets of objects and ignores line order unless you add -SyncWindow 0.

★ **Search text in files**	`findstr "error" log.txt`
	`findstr /S /I "text" *.*` *(recursive, ignore case)*
	`findstr /N "text" file.txt` *(line numbers)*
		`Select-String -Path log.txt -Pattern error` *(alias: sls)*
		`sls error *.log -CaseSensitive`
		`gci -r *.log | sls error`
			`grep "error" log.txt`
			`grep -ri "error" /var/log/` *(recursive, ignore case)*
			`grep -n "text" file.txt` *(line numbers)*	

Task	CMD	PowerShell	Bash	Notes

File permissions	`icacls file.txt`
	`icacls file.txt /grant User:F`
		`Get-Acl file.txt`
		`Set-Acl file.txt -AclObject $acl`
		`icacls file.txt` *(works here too)*
			`chmod 755 script.sh`
			`chmod +x script.sh`
			`chown user:group file.txt`
				🔒 Changing permissions on files you do not own needs admin or sudo. Windows uses ACLs; Unix uses rwx bits.

File attributes	`attrib +r file.txt` *(read-only)* `attrib +h file.txt` *(hidden)* `attrib -r -h file.txt`	`Set-ItemProperty file.txt IsReadOnly $true` `(Get-Item file.txt).Attributes`	`chattr +i file.txt` *(immutable, Linux)* `chattr -i file.txt` `chflags hidden file.txt` *(macOS)*	Unix has no hidden attribute: files whose names start with a dot are hidden by convention.
Symbolic links	`mklink link.txt C:\target.txt` `mklink /D linkdir C:\target`	`New-Item -ItemType SymbolicLink -Path link -Target target`	`ln -s target link` `ln -s /path/to/dir linkdir`	🔒 Windows needs an admin prompt or Developer Mode to create symlinks. Bash argument order is target first, then link.

Compress / archive	`tar -a -cf archive.zip folder` *(Windows 10 1803+)*
	`compact /C file.txt` *(NTFS compression, not an archive)*
		`Compress-Archive -Path folder -DestinationPath archive.zip`	`tar -czvf archive.tar.gz folder/`
			`zip -r archive.zip folder/`
			`gzip file.txt` *(replaces file with file.txt.gz)*
				bsdtar ships with Windows 10 and later and works in both CMD and PowerShell. It reads .zip, .tar.gz, .7z and more.

Extract	`tar -xf archive.zip`
	`tar -xf archive.tar.gz`
	`expand archive.cab C:\dest` *(CAB only)*
		`Expand-Archive archive.zip -DestinationPath dest\`	`unzip archive.zip`
			`tar -xzvf archive.tar.gz`
			`gunzip file.txt.gz`	

File hash / checksum	`certutil -hashfile file.iso SHA256`	`Get-FileHash file.iso` *(SHA256 by default)*
		`Get-FileHash file.iso -Algorithm MD5`
			`sha256sum file.iso`
			`md5sum file.iso`
			`shasum -a 256 file.iso` *(macOS)*	

Open with default app	`start report.pdf`
	`start .` *(Explorer here)*
	`start https://example.com`
		`Invoke-Item report.pdf` *(alias: ii)*
		`start .` *(alias of Start-Process)*
			`xdg-open report.pdf` *(Linux)*
			`open report.pdf` *(macOS)*
			`open .` *(Finder here, macOS)*	

Clear screen	`cls`	`Clear-Host` *(alias: cls, clear)* `Ctrl+L`	`clear` `Ctrl+L`	

Disk and Storage

Task	CMD	PowerShell	Bash	Notes

Free disk space	`fsutil volume diskfree C:`
	`wmic logicaldisk get name,size,freespace` *(wmic is deprecated)*
		`Get-PSDrive -PSProvider FileSystem`
		`Get-Volume`
		`Get-CimInstance Win32_LogicalDisk | Select DeviceID,Size,FreeSpace`
			`df -h`
			`df -h /home` *(one filesystem)*
				wmic is deprecated and removed from recent Windows 11 builds. Prefer Get-CimInstance.

Task	CMD	PowerShell	Bash	Notes

Disk and partition info	`diskpart` *(interactive: list disk, list volume)*
	`wmic diskdrive get model,size` *(deprecated)*
		`Get-Disk`
		`Get-Partition`
		`Get-Volume`
			`lsblk` *(Linux)*
			`sudo fdisk -l` *(Linux)*
			`diskutil list` *(macOS)*	

Folder size	`dir /S C:\folder` *(total at the end)*	`(Get-ChildItem -Recurse folder | Measure-Object Length -Sum).Sum / 1MB`	`du -sh folder/`
			`du -h --max-depth=1 . | sort -h` *(Linux)*
			`du -sh * | sort -h`	

Check disk	`chkdsk C:` `chkdsk C: /F` *(fix errors; system drive needs a reboot)*	`Repair-Volume -DriveLetter C -Scan` `Repair-Volume -DriveLetter C -OfflineScanAndFix`	`sudo fsck /dev/sda1` *(unmount first)* `diskutil verifyVolume /` *(macOS)*	🔒 Requires admin or sudo. Never run fsck on a mounted filesystem.
Format drive	`format D: /FS:NTFS /Q`	`Format-Volume -DriveLetter D -FileSystem NTFS`	`sudo mkfs.ext4 /dev/sdb1` `diskutil eraseDisk APFS Name disk2` *(macOS)*	⚠⚠ Destroys everything on the volume. Double-check the drive letter or device name first.

Mount / map drives	`net use Z: \\server\share` *(map network drive)*
	`net use Z: /delete`
	`mountvol` *(list volume mount points)*
		`New-PSDrive -Name Z -PSProvider FileSystem -Root \\server\share -Persist`
		`Mount-DiskImage file.iso`
		`Dismount-DiskImage file.iso`
			`sudo mount /dev/sdb1 /mnt/usb`
			`sudo umount /mnt/usb`
			`mount` *(list mounts)*	

Windows Repair

Task	Command	Notes
Repair system files (SFC)	`sfc /scannow` *(scan and repair protected system files)* `sfc /verifyonly` *(scan only, no repair)* `sfc /scannow /offbootdir=C:\ /offwindir=C:\Windows` *(offline, from recovery)*	🔒 Run from an elevated prompt. If SFC reports files it could not fix, run the DISM RestoreHealth step below, then run sfc /scannow again.
Repair the component store (DISM)	`DISM /Online /Cleanup-Image /CheckHealth` *(quick flag check)* `DISM /Online /Cleanup-Image /ScanHealth` *(full scan, no repair)* `DISM /Online /Cleanup-Image /RestoreHealth` *(download and repair)*	🔒 Elevated. RestoreHealth fixes the store SFC copies from, so run it first when files are corrupt. It uses Windows Update; add /Source:wim:X:\sources\install.wim:1 /LimitAccess to repair from an install image offline.
Check disk (CHKDSK)	`chkdsk C:` *(read-only report)* `chkdsk C: /F` *(fix file-system errors)* `chkdsk C: /R` *(find bad sectors, implies /F)*	🔒 Elevated. The system drive cannot be locked while Windows is running, so the check is scheduled for the next reboot. Also listed under Disk and Storage.
System Restore	`rstrui` *(open the System Restore wizard)* `Get-ComputerRestorePoint` *(PowerShell: list restore points)* `Checkpoint-Computer -Description "Before change"` *(PowerShell: create one)*	🔒 Creating or restoring needs admin. System Protection must be turned on for the drive first (System Properties > System Protection).

Task	Command	Notes
Boot to recovery options	`shutdown /r /o /t 0` *(reboot into Advanced Startup)* `reagentc /info` *(show recovery environment status)*	Advanced Startup leads to Startup Repair, Safe Mode, System Restore and a command prompt. Or hold Shift while clicking Restart.

System and User Information

Task	CMD	PowerShell	Bash	Notes

Current user	`whoami`
	`echo %USERNAME%`
	`whoami /groups`
		`whoami`
		`$env:USERNAME`
		`[Environment]::UserName`
			`whoami`
			`echo $USER`
			`id` *(uid, gid and groups)*	

Hostname	`hostname` `echo %COMPUTERNAME%`	`hostname` `$env:COMPUTERNAME`	`hostname` `hostnamectl` *(Linux with systemd)* `scutil --get ComputerName` *(macOS)*	

System information	`systeminfo`
	`systeminfo | findstr /B /C:"OS Name" /C:"OS Version"`
		`Get-ComputerInfo`
		`Get-ComputerInfo | Select OsName,OsVersion,CsTotalPhysicalMemory`
		`Get-CimInstance Win32_OperatingSystem`
			`uname -a`
			`cat /etc/os-release` *(Linux)*
			`hostnamectl` *(Linux)*
			`sw_vers` *(macOS)*
				Get-ComputerInfo is slow (several seconds). Pick properties with Select-Object.

OS version	`ver`	`[Environment]::OSVersion` `$PSVersionTable.PSVersion` *(PowerShell version)*	`uname -r` *(kernel)* `lsb_release -a` *(Debian/Ubuntu)* `sw_vers -productVersion` *(macOS)*	
CPU	`echo %NUMBER_OF_PROCESSORS%` `wmic cpu get name,numberofcores` *(deprecated)*	`Get-CimInstance Win32_Processor | Select Name,NumberOfCores,NumberOfLogicalProcessors`	`lscpu` *(Linux)* `nproc` *(Linux)* `sysctl -n machdep.cpu.brand_string` *(macOS)*	
Memory	`systeminfo | findstr /C:"Total Physical Memory" /C:"Available Physical Memory"`	`Get-CimInstance Win32_OperatingSystem | Select TotalVisibleMemorySize,FreePhysicalMemory` `Get-CimInstance Win32_PhysicalMemory | Select Capacity,Speed`	`free -h` *(Linux)* `cat /proc/meminfo` *(Linux)* `vm_stat` *(macOS)* `top` *(any)*	
Uptime	`systeminfo | findstr /C:"System Boot Time"` `net statistics workstation | findstr "since"`	`(Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime` `Get-Uptime` *(PowerShell 6+ only)*	`uptime` `uptime -p` *(pretty, Linux)* `uptime -s` *(boot time, Linux)*	

Task	CMD	PowerShell	Bash	Notes

Date and time	`date /T`
	`time /T`
	`echo %DATE% %TIME%`
		`Get-Date`
		`Get-Date -Format "yyyy-MM-dd HH:mm:ss"`
		`Get-Date -UFormat "%Y-%m-%d"`
			`date`
			`date +"%Y-%m-%d %H:%M:%S"`
			`date -u` *(UTC)*
				Without /T, CMD's date and time commands prompt you to change the clock (🔒 admin).

Logged-in users	`query user` *(alias: quser)*	`query user` `Get-CimInstance Win32_ComputerSystem | Select UserName`	`who` `w` *(who, plus what they are running)* `last` *(login history)*	

Processes, Services and Power

Task	CMD	PowerShell	Bash	Notes

★ **List processes**	`tasklist`
	`tasklist /V` *(verbose)*
	`tasklist /FI "IMAGENAME eq chrome.exe"`
		`Get-Process` *(alias: ps, gps)*
		`Get-Process -Name chrome`
		`Get-Process | Sort-Object CPU -Descending | Select -First 10`
			`ps aux`
			`ps aux | grep nginx`
			`top` *(live)*
			`htop` *(live, nicer; needs install)*	

★ **Kill process**	`taskkill /IM notepad.exe`
	`taskkill /PID 1234 /F` *(force)*
	`taskkill /F /IM chrome.exe /T` *(with child processes)*
		`Stop-Process -Name notepad` *(alias: kill)*
		`Stop-Process -Id 1234 -Force`
			`kill 1234` *(SIGTERM, polite)*
			`kill -9 1234` *(SIGKILL, force)*
			`pkill -f pattern`
			`killall firefox`
				⚠ Force-killing skips cleanup and can corrupt files being written. Try the polite form first.

Find process using a port	`netstat -ano | findstr :8080` *(PID in last column)*	`Get-NetTCPConnection -LocalPort 8080 | Select OwningProcess` `Get-Process -Id (Get-NetTCPConnection -LocalPort 8080).OwningProcess`	`sudo lsof -i :8080` `sudo ss -ltnp | grep 8080` *(Linux)*	

Run in background	`start /B program.exe` *(same window)*
	`start "" notepad.exe` *(new window)*
		`Start-Process notepad`
		`Start-Job { long-task }; Get-Job; Receive-Job 1`
		`command &` *(PowerShell 6+)*
			`command &`
			`nohup command &` *(survives logout)*
			`jobs / fg / bg`
			`Ctrl+Z` *(suspend the foreground job)*	

List services	`sc query` *(running)*
	`sc query state= all`
	`net start` *(running only)*
		`Get-Service`
		`Get-Service | Where Status -eq Running`
		`Get-Service -Name Spooler`
			`systemctl list-units --type=service` *(Linux)*
			`systemctl list-units --type=service --state=running`
			`launchctl list` *(macOS)*	

Task	CMD	PowerShell	Bash	Notes

Start / stop / restart service	`sc start Spooler`
	`sc stop Spooler`
	`net stop Spooler && net start Spooler`
	(net start/stop also work)
		`Start-Service Spooler`
		`Stop-Service Spooler`
		`Restart-Service Spooler`
			`sudo systemctl start nginx`
			`sudo systemctl stop nginx`
			`sudo systemctl restart nginx`
			`sudo systemctl enable nginx` *(start at boot)*
				🔒 Requires admin or sudo. sc and Get-Service use the service name (Spooler), not the display name (Print Spooler).

Service status	`sc query Spooler`
	`sc qc Spooler` *(configuration)*
		`Get-Service Spooler`
		`(Get-Service Spooler).Status`
			`systemctl status nginx`
			`systemctl is-active nginx`
			`journalctl -u nginx -f` *(follow logs, Linux)*	

Scheduled tasks	`schtasks /query`
	`schtasks /create /tn Backup /tr C:\backup.bat /sc daily /st 02:00`
	`schtasks /delete /tn Backup`
		`Get-ScheduledTask`
		`Get-ScheduledTask Backup | Start-ScheduledTask`
		`Register-ScheduledTask` *(see the Get-Help examples)*
			`crontab -l` *(list)*
			`crontab -e` *(edit)*
			`0 2 * * * /path/backup.sh` *(daily at 02:00)*
			`launchctl` *(macOS launchd)*	

Restart system	`shutdown /r /t 0`
	`shutdown /a` *(abort a pending shutdown)*
		`Restart-Computer`
		`Restart-Computer -Force`
			`sudo reboot`
			`sudo shutdown -r now`
			`sudo shutdown -r +5` *(in 5 minutes)*
				🔒 Requires admin or sudo.

Shut down system	`shutdown /s /t 0`
	`shutdown /s /t 60` *(in 60 seconds)*
	`shutdown /h` *(hibernate)*
		`Stop-Computer`
		`Stop-Computer -Force`
			`sudo shutdown -h now`
			`sudo poweroff`
			`sudo shutdown -c` *(cancel, Linux)*
				🔒 Requires admin or sudo.

Networking and Diagnostics

Task	CMD	PowerShell	Bash	Notes

★ **Ping host**	`ping google.com` *(4 packets)*
	`ping -n 10 192.168.1.1`
	`ping -t host` *(until Ctrl+C)*
		`Test-Connection google.com`
		`Test-Connection 8.8.8.8 -Count 5`
		`Test-Connection host -Quiet` *(True/False)*
			`ping google.com` *(until Ctrl+C)*
			`ping -c 5 google.com`
				Windows ping stops after 4 packets by default. Unix ping runs until you stop it.

★ **IP configuration**	`ipconfig`
	`ipconfig /all` *(MAC, DNS, DHCP)*
	`ipconfig /release && ipconfig /renew`
		`Get-NetIPConfiguration`
		`Get-NetIPAddress -AddressFamily IPv4`
			`ip addr` *(Linux; short form: ip a)*
			`ifconfig` *(macOS; deprecated on Linux)*
			`hostname -I` *(Linux, addresses only)*	

Public IP	`curl ifconfig.me`	`Invoke-RestMethod ifconfig.me`	`curl ifconfig.me`	
Connections and open ports	`netstat -an` *(all)* `netstat -ano` *(with PIDs)* `netstat -b` *(with program names, 🔒 admin)*	`Get-NetTCPConnection` `Get-NetTCPConnection -State Listen`	`ss -tulpn` *(Linux, preferred)* `sudo netstat -tulpn` *(older Linux)* `sudo lsof -i -P` *(macOS and Linux)*	netstat and ifconfig are deprecated on Linux in favour of ss and ip, but still work on macOS.

Task	CMD	PowerShell	Bash	Notes

DNS lookup	`nslookup google.com`
	`nslookup google.com 8.8.8.8` *(ask a specific server)*
	`nslookup -type=MX google.com`
		`Resolve-DnsName google.com`
		`Resolve-DnsName google.com -Type MX`
		`Resolve-DnsName google.com -Server 8.8.8.8`
			`dig google.com`
			`dig +short google.com`
			`dig MX google.com`
			`host google.com`	

Trace route	`tracert google.com`
	`tracert -d google.com` *(skip DNS, faster)*
	`pathping google.com`
		`Test-NetConnection google.com -TraceRoute`	`traceroute google.com`
			`tracepath google.com` *(Linux, no root needed)*
			`mtr google.com` *(live; needs install)*	

Routing table	`route print` `route add 192.168.2.0 mask 255.255.255.0 192.168.1.1`	`Get-NetRoute` `New-NetRoute -DestinationPrefix 192.168.2.0/24 -NextHop 192.168.1.1 -InterfaceIndex 12`	`ip route` *(Linux)* `sudo ip route add 192.168.2.0/24 via 192.168.1.1` `netstat -rn` *(macOS)*	🔒 Adding routes needs admin or sudo.
Flush DNS cache	`ipconfig /flushdns` `ipconfig /displaydns`	`Clear-DnsClientCache` `Get-DnsClientCache`	`sudo resolvectl flush-caches` *(Linux, systemd-resolved)* `sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder` *(macOS)*	
ARP table	`arp -a`	`Get-NetNeighbor` `Get-NetNeighbor -State Reachable`	`ip neigh` *(Linux)* `arp -a` *(macOS and Linux)*	

Download file	`curl -O https://example.com/file.zip`
	`curl -L -o out.zip URL` *(follow redirects, custom name)*
	`certutil -urlcache -split -f URL file.zip` *(legacy)*
		`Invoke-WebRequest URL -OutFile file.zip` *(alias: iwr)*
		`Invoke-RestMethod URL (parses JSON)` *(alias: irm)*
		`curl.exe -O URL`
			`wget URL`
			`curl -O URL`
			`curl -L -o out.zip URL`
				In Windows PowerShell 5.1, curl is an alias of Invoke-WebRequest. Type curl.exe to get the real tool.

Test a port	`curl -v telnet://host:443`
	`telnet host 80` *(optional feature, off by default)*
		`Test-NetConnection host -Port 443` *(alias: tnc)*
		`tnc host -Port 443 -InformationLevel Quiet`
			`nc -zv host 443`
			`curl -v telnet://host:443`
			`(echo > /dev/tcp/host/443) && echo open` *(Bash only)*	

SSH and remote copy	`ssh user@host`
	`scp file.txt user@host:/path/`
	`ssh -i key.pem user@host`
		`ssh user@host`
		`scp file.txt user@host:/path/`
		`Enter-PSSession -ComputerName host` *(WinRM, Windows to Windows)*
			`ssh user@host`
			`scp file.txt user@host:/path/`
			`rsync -avz src/ user@host:/dest/`
				The OpenSSH client is built into Windows 10 1809 and later.

Task	CMD	PowerShell	Bash	Notes
Wi-Fi profiles and passwords	`netsh wlan show profiles` *(saved networks)* `netsh wlan show profile name="MyWiFi" key=clear` *(reveals the password)* `netsh wlan connect name="MyWiFi"`	`netsh wlan show profiles` `netsh wlan show profile name="MyWiFi" key=clear | sls Key` *(just the password line)*	`nmcli device wifi list` *(Linux)* `sudo nmcli -s -g 802-11-wireless-security.psk connection show "MyWiFi"` *(Linux password)* `security find-generic-password -wa "MyWiFi"` *(macOS; prompts for keychain)*	The Windows password shows as "Key Content" in the output. On Linux the psk read needs sudo; macOS prompts for your login keychain.

Firewall	`netsh advfirewall show allprofiles`
	`netsh advfirewall firewall add rule name="App" dir=in action=allow program="C:\app.exe" enable=yes`
	`netsh advfirewall set allprofiles state on`
		`Get-NetFirewallProfile`
		`New-NetFirewallRule -DisplayName "App" -Direction Inbound -Action Allow -Program "C:\app.exe"`
		`Set-NetFirewallProfile -All -Enabled True`
			`sudo ufw status`
			`sudo ufw allow 22/tcp` *(Debian/Ubuntu)*
			`sudo firewall-cmd --add-port=8080/tcp --permanent` *(Fedora/RHEL)*
			`sudo pfctl -sr` *(macOS rules)*
				🔒 Requires admin or sudo. ufw is the friendly front end on Ubuntu; firewalld (firewall-cmd) on Fedora/RHEL; macOS uses pf plus the Application Firewall in System Settings.

Environment, History and Shell Basics

Task	CMD	PowerShell	Bash	Notes

★ **Show environment variables**	`set`
	`set PATH` *(all starting with PATH)*
	`echo %PATH%`
		`Get-ChildItem Env:` *(or: dir env:)*
		`$env:PATH`
		`$env:PATH -split ';'` *(one per line)*
			`printenv`
			`env`
			`echo $PATH`
			`echo $PATH | tr ':' '\n'` *(one per line)*	

Set environment variable	`set MYVAR=value` *(this session)* `setx MYVAR value` *(permanent, new windows only)* `setx PATH "%PATH%;C:\tools"` *(⚠ truncates at 1024 chars)*	`$env:MYVAR = 'value'` *(this session)* `[Environment]::SetEnvironmentVariable('MYVAR','value','User')` *(permanent)*	`export MYVAR=value` *(this session)* `MYVAR=value command` *(one command only)* `add the export line to ~/.bashrc or ~/.zshrc` *(permanent)*	Permanent changes only affect shells opened afterwards.

★ **Command history**	`doskey /history`
	`F7` *(popup list)*
		`Get-History` *(alias: h, history)*
		`Get-Content (Get-PSReadLineOption).HistorySavePath` *(all sessions)*
			`history`
			`history 20` *(last 20)*
			`history | grep ssh`	

Re-run a command	`F3 or ↑` *(previous)*	`↑` *(previous)* `Invoke-History 5` *(alias: r 5)*	`!!` *(previous)* `!5` *(history entry 5)* `!ssh` *(last command starting with ssh)* `sudo !!` *(repeat with sudo)*	

Task	CMD	PowerShell	Bash	Notes

Search history	`F7` *(dialog)*
	`type a prefix, then F8` *(cycle matches)*
		`Ctrl+R` *(reverse search)*
		`type a prefix, then F8`
		`Get-History | Where CommandLine -like '*git*'`
			`Ctrl+R` *(reverse search)*
			`history | grep pattern`	

Aliases	`doskey ll=dir /B $*` *(this session)*
	(persist via a startup script in the AutoRun registry key)
		`Set-Alias ll Get-ChildItem`
		`Get-Alias` *(list all)*
		`function gs { git status }` *(aliases cannot take arguments; use a function)*
			`alias ll='ls -la'`
			`alias` *(list all)*
			`unalias ll`	

Shell profile / startup	`HKCU\Software\Microsoft\Command Processor\AutoRun` *(registry value)*	`$PROFILE` *(path to your profile script)* `notepad $PROFILE` `. $PROFILE` *(reload)*	`~/.bashrc` *(interactive Bash)* `~/.bash_profile` *(login shells)* `~/.zshrc` *(zsh; default on macOS)* `source ~/.bashrc` *(reload)*	Put aliases, functions and PATH changes in the profile so they survive between sessions.
Exit code of last command	`echo %ERRORLEVEL%` `if %ERRORLEVEL% neq 0 echo failed`	`$?` *(True or False)* `$LASTEXITCODE` *(from the last native .exe)*	`echo $?` *(0 means success)*	

Chain commands	`a && b` *(b only if a succeeds)*
	`a || b` *(b only if a fails)*
	`a & b` *(both, regardless)*
		`a; b` *(both, regardless)*
		`a && b, a || b` *(PowerShell 7+ only)*
		(-and / -or are boolean operators, not chaining)
			`a && b`
			`a || b`
			`a; b`	

★ Get help	`dir /?`
	`help copy`
		`Get-Help Get-Process -Examples`
		`Get-Help *-Service` *(search by name)*
		`Get-Command -Noun Service`
		`Update-Help` *(once, 🔒 admin)*
			`man ls`
			`ls --help`
			`tldr ls` *(needs install)*	

Run as admin / root	`runas /user:Administrator cmd` *(or right-click the shell and choose Run as administrator)*	`Start-Process powershell -Verb RunAs` `sudo command` *(Windows 11 24H2+; enable in Settings > System > For developers)*	`sudo command` `sudo -i` *(root shell)* `su -` *(switch user)*	

Text Processing and Output

Task	CMD	PowerShell	Bash	Notes

Display text	`echo Hello World`
	`echo %PATH%`
	`echo.` *(blank line)*
		`Write-Output 'Hello'` *(alias: echo, write)*
		`Write-Host 'Text' -ForegroundColor Red` *(console only, not pipeable)*
		`"Value: $var"` *(double quotes expand variables)*
			`echo "Hello World"`
			`printf "%s\n" "text"`
			`echo -e "a\tb"` *(interpret escapes)*	

View a page at a time	`more file.txt`
	`dir /S | more`
		`Get-Content file.txt | more`
		`Get-Content file.txt | Out-Host -Paging`
			`less file.txt` *(q quits, / searches)*
			`more file.txt`
			`cat file.txt | less`	

Task	CMD	PowerShell	Bash	Notes
First lines of a file	`more +5 file.txt` *(skip the first 5 instead)* *(none; use PowerShell)*	`Get-Content file.txt -Head 10` `Get-Content file.txt | Select-Object -First 10`	`head file.txt` *(10 lines)* `head -n 20 file.txt` `head -c 100 file.bin` *(bytes)*	

Last lines of a file	*(none; use PowerShell)*	`Get-Content file.txt -Tail 10`
		`Get-Content log.txt -Wait` *(follow, like tail -f)*
			`tail file.txt`
			`tail -n 20 file.txt`
			`tail -f log.txt` *(follow)*	

Count lines / words	`find /c /v "" < file.txt` *(lines)* `type file.txt | find /c "error"` *(matching lines)*	`Get-Content file.txt | Measure-Object -Line -Word -Character` `(Get-Content file.txt).Count` *(lines)*	`wc -l file.txt` *(lines)* `wc -w file.txt` *(words)* `grep -c error file.txt` *(matching lines)*	

Sort lines	`sort file.txt`
	`sort /R file.txt` *(reverse)*
	`dir /B | sort`
		`Get-Content file.txt | Sort-Object`
		`Sort-Object -Unique`
		`Get-Process | Sort-Object CPU -Descending`
			`sort file.txt`
			`sort -r (reverse), -n (numeric), -u` *(unique)*
			`sort -k2 file.txt` *(by second field)*	

Unique lines	*(none; use PowerShell)*	`Get-Content file.txt | Sort-Object -Unique` `Get-Content file.txt | Group-Object | Sort Count -Desc` *(with counts)*	`sort -u file.txt` `sort file.txt | uniq -c` *(with counts)* `uniq file.txt` *(adjacent duplicates only)*	

Filter a stream	`dir | find "txt"`
	`tasklist | findstr /I chrome`
		`Get-Process | Where-Object Name -like 'chrome*'` *(alias: where, ?)*
		`Get-Process | Where CPU -gt 100`
		`gc log.txt | sls error`
			`ps aux | grep chrome`
			`grep -v debug log.txt` *(invert match)*
				PowerShell pipes objects, not text: filter on properties with Where-Object, then shape output with Select-Object or Format-Table.

Find and replace	*(none; use PowerShell)*	`(Get-Content file.txt) -replace 'old','new' | Set-Content file.txt` `'text' -replace '(\d+)','[$1]'` *(regex by default)*	`sed 's/old/new/g' file.txt` *(prints result)* `sed -i 's/old/new/g' file.txt` *(in place, Linux)* `sed -i '' 's/old/new/g' file.txt` *(in place, macOS)*	The parentheses around Get-Content read the whole file first so PowerShell can safely overwrite it.

Extract columns / fields	`for /f "tokens=2 delims=," %i in (file.csv) do @echo %i`
	(inside a .bat file use %%i)
		`Import-Csv data.csv | Select-Object Name,Email`
		`'a,b,c' -split ','`
		`Get-Content f.txt | ForEach { ($_ -split '\s+')[1] }`
			`cut -d, -f2 file.csv`
			`awk '{print $2}' file.txt`
			`awk -F: '{print $1}' /etc/passwd`	

Task	CMD	PowerShell	Bash	Notes

Shape and export output	`dir /B /O-D` *(bare, newest first)*
	`tasklist /FO CSV`
	`tasklist /FO CSV > procs.csv`
		`Get-Process | Select-Object Name,CPU`
		`Get-Process | Format-Table -AutoSize`
		`Get-Process | Format-List *`
		`Get-Process | Export-Csv procs.csv -NoTypeInformation`
		`Get-Process | ConvertTo-Json`
		`Get-Service | Out-GridView`
			`column -t file.txt` *(align columns)*
			`ls -l | awk '{print $9, $5}'`
			`jq . file.json` *(needs install)*
				Format-* cmdlets must be last in a pipeline; their output is for the screen only.

Clipboard	`dir | clip`	`Get-Process | Set-Clipboard` *(alias: scb)* `Get-Clipboard` *(alias: gcb)* `ls | clip`	`ls | pbcopy (macOS), pbpaste` `ls | xclip -selection clipboard` *(Linux, X11)* `ls | wl-copy` *(Linux, Wayland)*	

Redirection and Piping

Task	CMD	PowerShell	Bash	Notes

Redirect output (overwrite) >	`dir > list.txt`	`Get-Process > processes.txt`
		`Get-Process | Out-File -Encoding utf8 p.txt`
		`Get-Process | Set-Content p.txt`
			`ls -la > listing.txt`	Windows PowerShell 5.1 writes UTF-16 with >. Use Out-File -Encoding utf8 for files other tools will read. PowerShell 7 defaults to UTF-8.

Redirect output (append) >>	`echo text >> file.txt`	`Get-Date >> log.txt` `Add-Content log.txt 'line'`	`echo "new line" >> file.txt`	
Redirect input <	`sort < unsorted.txt`	`Get-Content input.txt | Sort-Object` *(PowerShell has no < operator)*	`sort < unsorted.txt` `cat <<EOF ... EOF` *(here-document)* `cmd <<< "string"` *(here-string)*	
Pipe |	`dir | find "txt"` `type file.txt | more`	`Get-Process | Sort-Object CPU -Descending | Select -First 5` `Get-ChildItem | Where Length -gt 1MB`	`ls -la | grep txt` `cat file.txt | sort | uniq -c`	CMD and Bash pipe text. PowerShell pipes objects; $_ (or $PSItem) is the current object inside a script block.

Redirect errors 2>	`command 2> errors.txt`
	`command 2>&1` *(merge errors into output)*
		`command 2> errors.txt`
		`command 2>&1`
		`$ErrorActionPreference = 'Stop'` *(make errors terminate)*
			`command 2> errors.txt`
			`command 2>&1`	

Redirect everything	`command > out.txt 2>&1`	`command *> all.txt` `command 3>&1 (warnings), 4>&1 (verbose), 6>&1` *(information)*	`command &> out.txt` `command > out.txt 2>&1`	

Task	CMD	PowerShell	Bash	Notes

Suppress output	`command > nul`
	`command > nul 2>&1`
		`command | Out-Null`
		`command > $null`
		`$null = command` *(fastest)*
			`command > /dev/null`
			`command &> /dev/null`
			`command 2> /dev/null` *(errors only)*	

Tee (file and screen)	*(none; use PowerShell)*	`Get-Process | Tee-Object -FilePath p.txt` `Get-Process | Tee-Object -Variable procs | Select Name`	`ls | tee listing.txt` `ls | tee -a listing.txt` *(append)* `echo x | sudo tee /etc/file` *(write as root)*	

Command substitution	`for /f %i in ('date /T') do set today=%i`	`$today = Get-Date -Format yyyy-MM-dd`
		`Write-Output "Today: $(Get-Date)"`
		`Copy-Item a.txt "backup-$(Get-Date -f yyyyMMdd).txt"`
			`today=$(date +%F)`
			`echo "Today: $(date)"`
			`cp a.txt "backup-$(date +%F).txt"`	

Users and Groups

Task	CMD	PowerShell	Bash	Notes
List users	`net user` `net user alice` *(details)*	`Get-LocalUser` `Get-LocalUser | Where Enabled`	`cat /etc/passwd` *(Linux)* `getent passwd` `dscl . list /Users` *(macOS)*	
Add user	`net user alice * /add` *(prompts for password)*	`New-LocalUser alice -Password (Read-Host -AsSecureString)` `New-LocalUser alice -NoPassword`	`sudo adduser alice` *(interactive, Debian/Ubuntu)* `sudo useradd -m alice && sudo passwd alice`	🔒 Requires admin or sudo. Avoid typing passwords on the command line: they end up in history.
Delete user	`net user alice /delete`	`Remove-LocalUser alice`	`sudo userdel alice` `sudo userdel -r alice` *(also remove home dir)*	🔒 Requires admin or sudo.
Change password	`net user alice *` *(prompts)*	`Set-LocalUser alice -Password (Read-Host -AsSecureString)`	`passwd` *(your own)* `sudo passwd alice`	

List groups	`net localgroup`
	`net localgroup Administrators` *(members)*
		`Get-LocalGroup`
		`Get-LocalGroupMember Administrators`
			`groups` *(current user)*
			`getent group`
			`id alice`	

Add user to group	`net localgroup Administrators alice /add`	`Add-LocalGroupMember -Group Administrators -Member alice`	`sudo usermod -aG sudo alice` *(Debian/Ubuntu)* `sudo usermod -aG wheel alice` *(Fedora/RHEL)* *(log out and back in to apply)*	🔒 Requires admin or sudo. ⚠ Forgetting -a in usermod removes the user from all other groups.

Package Management

Task	CMD	PowerShell	Bash	Notes
List installed	`winget list`	`winget list` `choco list` *(Chocolatey)* `Get-Module -ListAvailable` *(PowerShell modules)*	`apt list --installed` *(Debian/Ubuntu)* `dnf list installed` *(Fedora/RHEL)* `brew list` *(macOS)* `pacman -Q` *(Arch)*	wmic product get name is deprecated and very slow; use winget instead.

Search	`winget search vscode`	`winget search vscode`
		`choco search vscode`
		`Find-Module PSReadLine`
			`apt search vscode`
			`dnf search vscode`
			`brew search vscode`	

Show package info	`winget show Git.Git`	`winget show Git.Git`	`apt show git`
			`dnf info git`
			`brew info git`	

Install	`winget install Git.Git`
	`winget install --id Git.Git -e` *(exact id)*
		`winget install Git.Git`
		`choco install git -y`
		`Install-Module PSReadLine` *(PowerShell module)*
			`sudo apt install git`
			`sudo dnf install git`
			`brew install git`
			`sudo pacman -S git`
				🔒 Usually needs admin or sudo. brew and per-user winget installs are the exception.

Update	`winget upgrade` *(list available)*
	`winget upgrade --all`
		`winget upgrade --all`
		`choco upgrade all`
		`Update-Module`
			`sudo apt update && sudo apt upgrade`
			`sudo dnf upgrade`
			`brew update && brew upgrade`
			`sudo pacman -Syu`
				apt update only refreshes the package index. apt upgrade does the installing.

Remove	`winget uninstall Git.Git`	`winget uninstall Git.Git`
		`choco uninstall git`
		`Uninstall-Module name`
			`sudo apt remove git` *(keeps config)*
			`sudo apt purge git` *(removes config)*
			`sudo apt autoremove` *(orphaned dependencies)*
			`brew uninstall git`	

Building PowerShell One-Liners

Windows gives you two shells that look almost identical but obey different rules: **Command Prompt** (`cmd.exe`) and **Windows PowerShell** 5.1. This section is about reading a one-liner, understanding each piece, and assembling your own, rather than copying blindly. The worked examples at the end are real commands broken down part by part.

---

1. Which shell am I in?

> Look at the prompt. `PS C:\Users\you>` is PowerShell; `C:\Users\you>` with no `PS` in front is Command Prompt. They share many names (`cd`, `dir`, `cls`) but the syntax underneath differs, so a line that works in one can fail in the other.

- Open PowerShell: Start menu, type **PowerShell**. Open Command Prompt: type **cmd**.
- Open either as administrator: right-click it and choose **Run as administrator**, or press Ctrl+Shift+Enter when launching it from the Start menu. The title bar then reads "Administrator".
- Hop between them: type `powershell` to enter PowerShell, or `cmd` to drop into Command Prompt. Type `exit` to go back.
- Run one Command Prompt command from PowerShell with `cmd /c "..."` (used in example E).

Rule of thumb: if a line uses `$variables`, pipes `|` into a cmdlet, or has `{ }` blocks, it is PowerShell. If it uses slash flags like `/S /Q`, it is Command Prompt style.

---

2. PowerShell building blocks

- `$name = value` stores a value. `$log = 'C:\...\out.txt'` holds a path you reuse later in the same line.
- `|` (the pipeline) sends each object from the command on the left to the command on the right. See **Redirection and Piping** for the table.
- `ForEach-Object { ... }` runs the block once per piped object; inside it `$_` is the current object, and `$_.Name` reads that object's Name property.
- `foreach ($x in $list) { ... }` is the loop form for when you already have a list, rather than a pipeline (example B).
- `;` runs several commands in order on one line, left to right. See **Chain commands** under Environment, History and Shell Basics.
- `{ ... }` is a script block: code passed around as a single unit, such as the body of ForEach-Object.
- `| Out-Null` throws output away when you only want the side effect. See **Suppress output**.

---

3. Quoting, and the $ trap

- Single quotes `'...'` are literal: nothing inside is interpreted.
- Double quotes `"..."` expand variables. `"$($_.Name)"` runs the expression `$($_.Name)` and drops the result into the string; the `$( )` tells PowerShell to evaluate the property instead of printing the whole object.
- ⚠ A `$` in a path must be inside single quotes. In double quotes or unquoted, PowerShell treats `$RECYCLE` as a variable, which is empty.

That last point bites in a real way. Unquoted, this path collapses:

```
E:\$RECYCLE.BIN\   ->   E:\.BIN\     (wrong: $RECYCLE read as an empty variable)
'E:\$RECYCLE.BIN'                       (right: single quotes keep the $ literal)
```

---

4. Aliases that lie: Command Prompt habits that break in PowerShell

PowerShell defines `dir`, `del` and `rmdir` as aliases, but they point to cmdlets with different switches, so the Command Prompt flags fail.

- `rmdir /S /Q folder` works in Command Prompt. In PowerShell `rmdir` is `Remove-Item`, which has no `/S /Q` and errors. The PowerShell form is `Remove-Item folder -Recurse -Force`.

- `dir` is `Get-ChildItem`; the Command Prompt hidden-files switch `dir /A` is `Get-ChildItem -Force`.
- `del` is `Remove-Item`; `del /Q` does nothing useful, use `-Force`.
- To deliberately use the Command Prompt version from PowerShell, wrap it: `cmd /c "rmdir /s /q folder"`.

The full alias list is in **Notes and Gotchas**, under "PowerShell is different".

---

5. Robocopy as a read-only inspector

Robocopy is a copy tool, but with `/L` it copies nothing and only reports what it would do. That makes it a fast way to list or size folders, including protected ones a normal account cannot read. ⚠ Without `/L` it really copies, so always include `/L` when you are only inspecting.

- `/L` list only: report, do not copy. The safety flag.
- `/E` include all subfolders, even empty ones.
- `/B` backup mode: read files the normal account cannot. 🔒 Needs administrator.
- `/BYTES` show sizes in bytes instead of rounded units.
- `/TS` add timestamps and `/FP` full paths to the listing.
- `/NC` no class, `/NDL` no directory list, `/NJH` no job header, `/NJS` no job summary, `/NFL` no file list: trim the output down to the lines you want.
- `/R:0` do not retry failed files and `/W:0` wait zero seconds: fail fast rather than hang on locked files.
- `/UNILOG:file` write a Unicode log to that file (paired with `$log` in example A).

Robocopy's exit codes are unusual: 0 means nothing to do, 1 means files were (or would be) copied, and anything below 8 is success. 8 or higher is a real error. So a non-zero exit from robocopy is normal, not a failure.

---

6. Filtering and shaping output

These cmdlets chain after a `|` to cut a firehose down to what you want. Each already has a reference row; here is when to reach for which.

- `Select-String 'pattern'` keeps only matching lines, like grep. Example B uses `Select-String 'Files :|Bytes :'` to keep just the summary lines. See **Search text in files**.
- `Where-Object { $_.X -gt 1 }` keeps objects that match a condition. See **Filter a stream**.
- `Select-Object A,B` picks columns; `Select-Object -First 10` takes the first rows.
- `Sort-Object Size -Descending` orders objects.
- `Format-Table -AutoSize` lays the result out as aligned columns (example C). Put it last; it is for display only. See **Shape and export output**.

7. Reading system and disk info

- `Get-PhysicalDisk` lists the physical drives; pipe each into `Get-StorageReliabilityCounter` for temperature, read and write error counts, and power-on hours (example A).
- `Get-CimInstance Win32_LogicalDisk` lists volumes with size and free space (example C). Also in **Disk and Storage**.
- `vssadmin list shadowstorage` shows the space used by Volume Shadow Copies and restore points. 🔒 Needs administrator.
- `icacls path` shows or changes the Windows permissions on a file or folder. See **File permissions**.

8. Safety habits

- **Look before you delete.** List the target first with `robocopy ... /L`, `dir`, or `Get-ChildItem`, and read it.
- **Preview, then apply.** Good scripts take a switch so the plain run only reports and a second run acts (example D).
- **Know what needs admin.** `/B`, `vssadmin`, and changing permissions or services all need an Administrator window (🔒).
- ⚠ **Some actions cannot be undone.** `Remove-Item -Recurse -Force`, `rd /s /q`, `del`, and formatting all bypass the Recycle Bin. There is no undo. Example E is shown to read, not to run.

Worked example: build example A step by step

Example A saves a file listing, then prints drive health. Grow it in stages.

Start by listing a folder without copying anything:

```
robocopy 'E:\found.000' 'C:\robocopy_list_only' /L /E
```

Trim the output, fail fast on locked files, and send it to a log held in a variable:

```
$log = 'C:\Users\randu\AppData\Local\Temp\found_E_listing.txt'
robocopy 'E:\found.000' 'C:\robocopy_list_only' /L /E /B /BYTES /TS /FP /NC /NDL /NJH /NJS /R:0 /W:0 /UNILOG:$log | Out-
Null
```

The `| Out-Null` hides robocopy's console chatter because you only want the log file. Now add the drive-health read after a `;`:

```
Get-PhysicalDisk | ForEach-Object { $c = $_ | Get-StorageReliabilityCounter; "$($_.DeviceId) $($_.FriendlyName)
temp=$($c.Temperature) readErrors=$($c.ReadErrorsUncorrected) writeErrors=$($c.WriteErrorsUncorrected)
hours=$($c.PowerOnHours)" }
```

For each physical disk, `$c` holds its reliability counters, and the double-quoted string pulls named fields out with `$( )`. Join the two halves with `;` and end with `'done'`, and that is the whole of example A.

More worked examples

B. Size several protected folders. A `foreach` loop runs robocopy on each path and keeps only the summary lines:

```
foreach ($p in 'E:\found.000','E:\System Volume Information','G:\found.000') { $p; robocopy $p 'C:\robocopy_list_only' /L
/E /B /BYTES /NFL /NDL /NJH /R:0 /W:0 | Select-String 'Files :|Bytes :' }; vssadmin list shadowstorage /for=E:
```

- `foreach ($p in a,b,c)` loops over the three paths; `$p;` prints the path as a header before its totals.
- `/NFL` (no file list) joins the trims from section 5, leaving just the totals; `Select-String 'Files :|Bytes :'` keeps those two lines.
- `/B` needs admin. `vssadmin list shadowstorage /for=E:` at the end reports shadow-copy usage on E:.

C. List drive sizes and free space as a clean table:

```
Get-CimInstance Win32_LogicalDisk | Select-Object DeviceID, VolumeName, @{n='SizeGB';e={[math]::Round($_.Size/1GB,1)}},
@{n='FreeGB';e={[math]::Round($_.FreeSpace/1GB,1)}} | Format-Table -AutoSize
```

- `@{n='SizeGB';e={...}}` is a calculated column: `n` is its name, `e` is the expression. Here it turns bytes into GB with `[math]::Round(.../1GB,1)`.
- `Format-Table -AutoSize` aligns it. Change `1GB` to `1MB`, or the rounding `1` to `2`, to reshape it.

D. Run a script preview-then-apply. Two runs of the same script:

```
powershell -ExecutionPolicy Bypass -File C:\path\to\script.ps1
powershell -ExecutionPolicy Bypass -File C:\path\to\script.ps1 -Apply
```

- `-ExecutionPolicy Bypass` lets this one script run without permanently changing your machine's policy. See **Allow PowerShell scripts** in Notes.
- The first line has no `-Apply`, so a well-written script only previews. The second adds `-Apply` to make the changes. Preview, read it, then apply.

E. The same delete in each shell, shown for explanation only:

```
Command Prompt:  rd /s /q "E:\$RECYCLE.BIN"
PowerShell:      cmd /c 'rd /s /q "E:\$RECYCLE.BIN"'
```

- `rd /s /q` is Command Prompt: `/s` recurse, `/q` quiet. It permanently deletes the folder and everything under it.
- In PowerShell, `rd` would be `Remove-Item` and would mis-read `$RECYCLE`, so the line wraps the real Command Prompt call in `cmd /c '...'`, using single quotes to protect the `$`.

- ⚠ This is permanent and cannot be undone. It is here to read, not to run.

Pattern: search the end of a log

A common need is to find recent events in a large log without reading the whole file. It composes the three cmdlets from section 6 into a reusable template:

```
Get-Content '<path to log>' -Tail <lines to read> | Select-String '<pattern>|<other pattern>' | Select-Object -Last
<matches to show>
```

A real example, pulling the last few Steam content events:

```
Get-Content 'E:\Program Files (x86)\Steam\logs\content_log.txt' -Tail 20000 | Select-String 'update started|Validation:
read' | Select-Object -Last 20
```

Built one stage at a time:

- `Get-Content 'file' -Tail 20000` reads only the last 20,000 lines, so a huge log stays fast. Pick a number big enough to reach back to when the event happened. Single quotes keep spaces and `$` in the path safe.
- `|` passes each line to the next command.
- `Select-String 'a|b'` keeps lines matching a or b. It is a regex, so `|` means OR, and it ignores case by default; add `-CaseSensitive` if case matters.
- `Select-Object -Last 20` keeps only the newest 20 matches, because logs are oldest-first. Without it you get every match.

Useful variations:

- Show surrounding lines: `Select-String 'error' -Context 2,2` (two before, two after).
- Plain text instead of regex: `Select-String -SimpleMatch 'C:\path (x86)'`.
- Exclude noise: `... | Where-Object { $_.Line -notmatch 'noise' }`.
- Text only, without the match formatting: `... | ForEach-Object { $_.Line }`.
- Search many files at once: `Select-String -Path 'C:\logs\*.log' -Pattern 'error' | Select-Object -Last 20`.
- Count matches rather than show them: `(Select-String -Path 'file' -Pattern 'error').Count`.
- Watch live: `Get-Content 'file' -Tail 20 -Wait | Select-String 'error'`, then Ctrl+C to stop.
- Save results: `... | Out-File results.txt`. ⚠ Out-File overwrites an existing file; add `-Append` to add to it instead.

The same idea in Bash, with the full walkthrough in the Building Bash One-Liners section:

```
tail -n 20000 'file.log' | grep -E 'update started|Validation: read' | tail -n 20
```

Gotchas:

- Regex special characters like `. ( ) [ ] \` need escaping with a backslash, or use `-SimpleMatch` to match them literally.
- `-Tail` counts lines and `-Last` counts matches, so the two numbers are not interchangeable.
- This is PowerShell only. Command Prompt has no tail-then-filter equivalent (its nearest tool is `findstr`); the Bash form is the line above. See the Building Bash One-Liners and Building Command Prompt One-Liners sections.
- Reading a log is safe while the program is still writing to it; nothing here changes the file. Only `Out-File` writes, and it overwrites unless you add `-Append`.

Building Bash One-Liners

Bash (Linux, macOS, WSL, and Git Bash on Windows) builds one-liners by piping small text tools together. Same idea as the PowerShell section, different pieces: Bash pipes **plain text**, not objects. The reference tables above hold the individual commands; this is about assembling them.

Building blocks

- `name=value` sets a variable, with **no spaces** around `=`. Read it as `$name` or `${name}`. `files=$(ls)` captures a command's output into one.
- `|` pipes the text output of the command on the left into the command on the right.
- `$(command)` substitutes a command's output inline; `$(( ))` does arithmetic.
- `;` runs commands in order; `&&` runs the next only if the previous succeeded; `||` only if it failed. See **Chain commands**.
- `for f in *.log; do ...; done` loops over items; `{ ...; }` groups commands.
- `>` redirects output, `>>` appends, `2>` redirects errors, `> /dev/null` throws output away. See **Redirection and Piping**.

Quoting and globbing

- Single quotes `'...'` are literal; double quotes `"..."` expand `$variables` and `$(...)`.
- The shell expands `*`, `?` and `[abc]` into matching filenames **before** the command runs. Quote the pattern (`'*.log'`) when the command itself should see it, as with `find` and `grep`.
- Always quote a variable that may contain spaces: `"$file"` survives a name like `My File.txt`; bare `$file` would split into two arguments.

Filtering and shaping text

Bash pipes plain text, so you compose small filters. Each has a reference row; reach for:

- `grep 'pattern'` keeps matching lines; `grep -E 'a|b'` for regex OR, `-i` ignore case, `-v` invert, `-r` recurse, `-c` count. See **Search text in files**.
- `cut -d, -f2` and `awk '{print $2}'` pull out columns. See **Extract columns / fields**.
- `sed 's/old/new/'` substitutes; `sort`, `uniq -c`, `head`, `tail`, `wc -l` round out the kit. See **Text Processing and Output**.

- `xargs` turns a list into arguments: `find . -name '*.tmp' | xargs rm`. Use `find -print0 | xargs -0` so names with spaces survive.

Pattern: search the end of a log

The Bash version of the PowerShell log-search pattern. Same shape, different tools:

```
tail -n <lines to read> '<path to log>' | grep -E '<pattern>|<other pattern>' | tail -n <matches to show>
```

A real example:

```
tail -n 20000 ~/.steam/steam/logs/content_log.txt | grep -E 'update started|Validation: read' | tail -n 20
```

- `tail -n 20000 file` reads only the last 20,000 lines, so a huge log stays fast.
- `grep -E 'a|b'` keeps lines matching a or b; `-E` turns on extended regex where `|` means OR. It is case-sensitive by default; add `-i` to ignore case.
- The final `tail -n 20` keeps the newest 20 matches, because logs are oldest-first.

Variations:

- Surrounding lines: `grep -C 2 'error'` (two each side); `-A`/`-B` for after/before only.
- Plain text, no regex: `grep -F 'C:\path (x86)'`.
- Exclude noise: `... | grep -v 'noise'`.
- Many files: `grep -rE 'error' /var/log/ | tail -n 20`.
- Count matches: `grep -c 'error' file`.
- Watch live: `tail -f file | grep --line-buffered 'error'`, then Ctrl+C to stop.

Gotchas:

- `tail -n` counts lines and the final `tail` counts matched lines, so the two numbers are not interchangeable, exactly as with PowerShell's `-Tail` and `-Last`.
- macOS ships BSD `tail`, `grep` and `sed`; `grep -E` works everywhere but some GNU-only flags differ. See **macOS, zsh and WSL** in Notes.
- Reading a log is safe while the program writes; only `>` and `tee` change files, and `>` overwrites unless you use `>>` or `tee -a`.

Safety habits

- **Look before you delete.** List first with `ls` or `find ... -print`, read it, then act.
- ⚠ `rm` **is permanent.** There is no Trash on the command line. `rm -i` prompts per file; `rm -rf` asks nothing and cannot be undone.
- **Preview, then apply.** Run a `find` without `-delete`, or a `sed` without `-i`, first and read the output.

- 🔒 System paths, other users' files, and most of `/etc` need `sudo`. See **Run as admin / root**.

Worked example: delete old logs safely

Grow a cleanup one-liner, previewing at every stage. First, just list what matches:

```
find /var/log -name '*.log' -mtime +30
```

Read the list, then total its size before committing:

```
find /var/log -name '*.log' -mtime +30 -print0 | du -ch --files0-from=- | tail -1
```

Only once you trust the list, add the delete. This cannot be undone:

```
find /var/log -name '*.log' -mtime +30 -delete
```

- `-mtime +30` matches files modified more than 30 days ago; `-print0` with `--files0-from=-` handles names containing spaces.
- ⚠ `-delete` is permanent. Running the `find` without it first, and reading the list, is the whole safety habit.

Building Command Prompt One-Liners

Yes, Command Prompt (`cmd.exe`) does one-liners, but with a small toolset and quirky syntax. It pipes plain text between a handful of built-ins and has no objects, no `$(...)` substitution, and little string handling. For anything richer, use PowerShell (see **Building PowerShell One-Liners**). This covers what CMD does well and where it stops.

Building blocks

- `set VAR=value` sets a variable, **no spaces** around `=`; read it as `%VAR%`. Inside a `for` loop or a parenthesised block, `%VAR%` is read once up front, so use delayed expansion `!VAR!` after `setlocal enabledelayedexpansion`.
- `&` runs the next command regardless; `&&` runs it only if the previous succeeded; `||` only if it failed.
- `|` pipes text between commands, but only a few understand it: `find`, `findstr`, `sort`, `more`.
- `>` redirects output, `>>` appends, `2>` redirects errors, `2>&1` merges them, `>nul` throws output away.
- There is no `$(...)`. To capture a command's output into a loop or variable, use `for /f` (below).

for /f: the workhorse

`for /f` loops over the lines of a file, a string, or another command's output. It is how CMD parses and composes.

- Over a file's fields: `for /f "tokens=2 delims=," %i in (file.csv) do @echo %i` prints the second comma-separated field of each line.
- Over a command's output: wrap the command in single quotes: `for /f "delims=" %i in ('dir /b *.txt') do @echo %i`.
- `tokens=` picks which fields, `delims=` sets the separators, `skip=1` skips a header line.
- At an interactive prompt use `%i`; inside a `.bat` file double it to `%%i`.

Filtering text

- `findstr` is CMD's grep: `findstr "error" log.txt`, with `/I` ignore case, `/S` recurse, `/V` invert, `/N` line numbers, `/R` regex, and `/C:"exact phrase"` for a literal phrase.
- `find` is simpler: `find "text"` keeps matching lines, `find /c /v ""` counts lines, `find /v "skip"` drops matching lines.
- `sort` orders lines and `more` pages them. There is no `cut`, `awk` or `sed`; reach for PowerShell when you need those. See **Text Processing and Output**.

Pattern: search a log

CMD has no `-Tail`/`-Last` equivalent, so it reads the whole file. For recent events, filter with `findstr` and page the result:

```
findstr /I "update started" content_log.txt | more
```

- `/I` ignores case; add more terms with `findstr /I "started errored"` (space-separated terms are ORed).
- `| more` pages the output; `> out.txt` saves it instead.
- ⚠ There is no built-in "last 20 lines". For the tail-then-filter pattern, use PowerShell `Get-Content -Tail` or Bash `tail` (see those sections).

Safety habits

- **Look before you delete.** List with `dir` first and read it.
- **Dry-run a loop** by putting `echo` in front of the action: `for ... do @echo del "%i"` prints what it *would* delete without deleting. Remove `echo` to act.
- ⚠ `del` and `rd /s /q` are permanent; there is no Recycle Bin from the command line and no undo.
- 🔒 System folders and other users' files need an Administrator prompt.

Worked example: delete matching files, dry-run first

Find every `.tmp` under the current folder and preview the deletes. The `@echo` makes it a dry run:

```
for /f "delims=" %i in ('dir /b /s *.tmp') do @echo del "%i"
```

Read the printed list. Only when you trust it, drop the `echo` to actually delete. This cannot be undone:

```
for /f "delims=" %i in ('dir /b /s *.tmp') do @del "%i"
```

- `dir /b /s *.tmp` lists bare, recursive full paths; `for /f` feeds each to the action.
- ⚠ The second line is permanent. The `@echo` dry run is the whole safety habit; always run it first.

Developer Setup

Task	Windows	macOS	Linux	Notes

Install WSL (Linux on Windows)	`wsl --install` *(Ubuntu by default; reboot afterwards)*
	`wsl --install -d Debian` *(pick a distro)*
	`wsl -l -v` *(list distros and versions)*
	`wsl --update`
	`wsl --shutdown`
	`wsl` *(open the default distro)*
		(not needed; macOS is already Unix)	*(not needed)*	🔒 Run from an administrator prompt. Inside WSL, Windows drives are under /mnt/c and Windows tools are on the PATH, so code . opens VS Code.

Package manager	`winget --version` *(built in on Windows 10 1709+ and 11)* `winget source update` *(Chocolatey is optional: see chocolatey.org/install)*	`/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"` `brew doctor`	*(apt, dnf or pacman is already installed)* `sudo apt update` *(refresh the index first)*	If winget is missing, install "App Installer" from the Microsoft Store.
Git	`winget install --id Git.Git -e` `git --version`	`xcode-select --install` *(Apple's git plus compilers)* `brew install git` *(newer version)*	`sudo apt install git` `sudo dnf install git`	On Windows this also installs Git Bash.
GitHub CLI	`winget install --id GitHub.cli -e` `gh auth login`	`brew install gh` `gh auth login`	`sudo apt install gh` *(Ubuntu 23.04+; older distros: see cli.github.com)* `gh auth login`	gh auth login signs in through the browser and stores credentials that git push uses too.

Task	Windows	macOS	Linux	Notes

SSH key for GitHub	`ssh-keygen -t ed25519 -C "you@example.com"`
	`Get-Service ssh-agent | Set-Service -StartupType Automatic; Start-Service ssh-agent` *(PowerShell, 🔒 admin)*
	`ssh-add $env:USERPROFILE\.ssh\id_ed25519` *(PowerShell)*
	`gh ssh-key add ~/.ssh/id_ed25519.pub`
		`ssh-keygen -t ed25519 -C "you@example.com"`
		`ssh-add --apple-use-keychain ~/.ssh/id_ed25519`
		`gh ssh-key add ~/.ssh/id_ed25519.pub`
			`ssh-keygen -t ed25519 -C "you@example.com"`
			`eval "$(ssh-agent -s)"; ssh-add ~/.ssh/id_ed25519`
			`gh ssh-key add ~/.ssh/id_ed25519.pub`
				Test with ssh -T git@github.com. The Windows ssh-agent service is off by default.

Node.js	`winget install OpenJS.NodeJS.LTS`
	`winget install Schniz.fnm` *(version manager)*
	`node -v; npm -v`
		`brew install node`
		`brew install fnm` *(version manager)*
			`curl -fsSL https://fnm.vercel.app/install | bash` *(version manager)*
			`fnm install --lts`
			(distro packages are often years old)
				A version manager such as fnm or nvm lets you switch Node versions per project.

Python	`winget install Python.Python.3.12`
	`python --version`
	`py -3` *(launcher; picks an installed version)*
		`brew install python`
		`python3 --version`
			`sudo apt install python3 python3-pip python3-venv`
			`python3 --version`
				Per-project environment: python -m venv .venv, then .venv\Scripts\activate (Windows) or source .venv/bin/activate.

VS Code	`winget install Microsoft.VisualStudioCode`
	`code .` *(open the current folder)*
		`brew install --cask visual-studio-code`
		`code .` *(first run Shell Command: Install code from the Command Palette)*
			`sudo snap install code --classic`
			(or the .deb / .rpm from code.visualstudio.com)
			`code .`	

PowerShell 7 and Windows Terminal	`winget install Microsoft.PowerShell`
	`winget install Microsoft.WindowsTerminal`
	`pwsh` *(start PowerShell 7)*
		`brew install --cask powershell`
		`pwsh`
			(see Microsoft's install page for your distro)
			`pwsh`
				Windows PowerShell 5.1 is built in. PowerShell 7 (pwsh) is the current cross-platform version and installs alongside it. Windows 11 already includes Windows Terminal.

Compilers and build tools	`winget install Microsoft.VisualStudio.2022.BuildTools` *(choose "Desktop development with C++" in the installer)*	`xcode-select --install`	`sudo apt install build-essential` `sudo dnf groupinstall "Development Tools"`	Needed by npm and pip packages that compile native code.

Docker	`winget install Docker.DockerDesktop`
	`docker run hello-world`
		`brew install --cask docker`
		`docker run hello-world`
			`curl -fsSL https://get.docker.com | sh`
			`sudo usermod -aG docker $USER` *(then log out and in)*
			`docker run hello-world`
				Docker Desktop on Windows runs on WSL 2, so install WSL first.

Task	Windows	macOS	Linux	Notes
Allow PowerShell scripts	`Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` *(PowerShell)* `Get-ExecutionPolicy -List`	*(not needed)*	*(not needed)*	Fixes "running scripts is disabled on this system". RemoteSigned runs local scripts and signed downloads only.

Check what is installed	`winget list`
	`git --version; node -v; python --version; code -v`
	`where git` *(path to the executable)*
		`brew list`
		`git --version; node -v; python3 --version`
		`which git`
			`apt list --installed 2>/dev/null | grep -i git`
			`git --version; node -v; python3 --version`
			`which git`
				After installing anything, open a new terminal window. Existing shells keep the old PATH.

Git

Task	Command	Notes

One-time setup	`git config --global user.name "Your Name"`
	`git config --global user.email "you@example.com"`
	`git config --global init.defaultBranch main`
	`git config --list` *(show all settings)*
		Settings live in ~/.gitconfig. Drop --global to set them for one repository only.

Start a repository	`git init`
	`git clone https://github.com/user/repo.git`
	`git clone --depth 1 URL` *(latest commit only, fast)*	

See what changed	`git status` `git status -s` *(short form)* `git diff` *(unstaged changes)* `git diff --staged` *(what will be committed)* `git diff main..feature` *(between branches)*	

History	`git log --oneline --graph --all`
	`git log -p file.txt` *(changes to one file)*
	`git log -5 --stat`
	`git show HEAD` *(last commit in full)*
	`git blame file.txt` *(who changed each line)*	

Stage and commit	`git add file.txt`
	`git add -A` *(everything)*
	`git add -p` *(pick hunks interactively)*
	`git commit -m "Message"`
	`git commit -am "Message"` *(add tracked files and commit)*
	`git commit --amend` *(fix the last commit)*
		⚠ Do not amend a commit you have already pushed; it rewrites history others may have.

Task	Command	Notes
Unstage or discard	`git restore --staged file.txt` *(unstage, keep changes)* `git restore file.txt` *(discard local changes)* `git clean -fd` *(delete untracked files and folders)* `git clean -n` *(dry run first)*	⚠ restore and clean throw work away with no undo. Run git clean -n first.
Branches	`git branch` *(list)* `git branch -a` *(include remote)* `git switch -c feature` *(create and switch)* `git switch main` `git branch -m old new` *(rename)* `git branch -d feature` *(delete merged)* `git branch -D feature` *(force delete)*	switch and restore replaced the overloaded checkout in Git 2.23. checkout still works everywhere.
Merge and rebase	`git merge feature` *(merge into current branch)* `git rebase main` *(replay current branch onto main)* `git rebase -i HEAD~3` *(squash or reorder last 3)* `git merge --abort / git rebase --abort`	Merge keeps history as it happened. Rebase makes it linear. ⚠ Never rebase commits that are already pushed and shared.

Remotes	`git remote -v`
	`git remote add origin https://github.com/user/repo.git`
	`git remote set-url origin NEW_URL`	

Push and pull	`git push -u origin main` *(first push; sets upstream)*
	`git push`
	`git pull` *(fetch and merge)*
	`git pull --rebase`
	`git fetch` *(download without merging)*
	`git push origin --delete feature` *(delete remote branch)*	

Undo commits	`git revert HEAD` *(new commit that undoes the last one; safe)* `git reset --soft HEAD~1` *(uncommit, keep changes staged)* `git reset --mixed HEAD~1` *(uncommit, keep changes unstaged)* `git reset --hard HEAD~1` *(uncommit and discard changes)* `git reflog` *(find lost commits)*	⚠ reset --hard deletes work. revert is the safe choice for anything already pushed. reflog can recover a reset for about 30 days.

Stash	`git stash` *(shelve changes)*
	`git stash push -m "wip"` *(with a name)*
	`git stash list`
	`git stash pop` *(restore latest and drop it)*
	`git stash apply stash@{1}`
	`git stash drop`	

Task	Command	Notes

Tags and releases	`git tag` *(list)*
	`git tag -a v1.0 -m "Release 1.0"`
	`git push --tags`
	`git checkout v1.0` *(look at a tag)*	

Ignore and untrack files	`git rm --cached file.txt` *(stop tracking, keep file)* `git rm -r --cached folder/` `git check-ignore -v file.txt` *(why is this ignored?)* `git mv old.txt new.txt` *(rename and stage)*	Patterns in .gitignore only affect files not yet tracked. Untrack committed files with git rm --cached first.
Search	`git grep "pattern"` *(search tracked files)* `git log -S "text"` *(commits that added or removed text)* `git log --grep "fix"` *(search commit messages)* `git log --author="Name"`	
Who did what	`git shortlog -sn` *(commits per author)* `git log --since="2 weeks ago"` `git bisect start / git bisect good v1.0 / git bisect bad` *(find the breaking commit)*	

Help	`git help commit`
	`git commit -h` *(short flag list)*
	`git <command> --help`	

Docker

Task	Command	Notes
List containers	`docker ps` *(running)* `docker ps -a` *(include stopped)* `docker ps -q` *(IDs only)*	On Linux, prefix with sudo or add yourself to the docker group. Docker Desktop on Windows and macOS runs the daemon for you.

List images	`docker images`
	`docker image ls`
	`docker history nginx` *(layers)*	

Run a container	`docker run -d -p 8080:80 --name web nginx` *(detached, port map, named)* `docker run -it ubuntu bash` *(interactive shell)* `docker run --rm alpine echo hi` *(remove when it exits)* `docker run -v ${PWD}:/app node` *(mount current folder)*	-d runs in the background, -p maps host:container ports, -v mounts a volume, --rm cleans up on exit.

Task	Command	Notes
Shell into a container	`docker exec -it web bash` `docker exec -it web sh` *(if bash is missing)* `docker attach web` *(attach to PID 1; Ctrl+P Ctrl+Q to detach)*	

Logs	`docker logs web`
	`docker logs -f web` *(follow)*
	`docker logs --tail 50 web`	

Stop, start, remove	`docker stop web`
	`docker start web`
	`docker restart web`
	`docker rm web` *(add -f to force)*
	`docker rm -f $(docker ps -aq)` *(remove all ⚠)*
		⚠ The last command removes every container.

Remove images	`docker rmi nginx`
	`docker image rm nginx:latest`
	`docker image prune` *(dangling)*
	`docker image prune -a` *(all unused)*	

Free up space	`docker system df` *(show usage)* `docker system prune` *(stopped containers, unused networks, dangling images)* `docker system prune -a --volumes` *(⚠ everything unused, including volumes)*	⚠ prune -a --volumes deletes data in unused volumes. Check docker volume ls first.

Build an image	`docker build -t myapp .`
	`docker build -t myapp:1.0 -f Dockerfile.prod .`
	`docker tag myapp myrepo/myapp:1.0`
		The trailing dot is the build context, the folder sent to the daemon.

Push and pull	`docker pull redis:7`
	`docker login`
	`docker push myrepo/myapp:1.0`	

Copy files	`docker cp web:/etc/nginx/nginx.conf ./nginx.conf` *(out of the container)* `docker cp ./file.txt web:/tmp/` *(into the container)*	
Inspect and stats	`docker inspect web` *(full JSON)* `docker stats` *(live CPU and memory)* `docker port web` `docker top web` *(processes)*	

Volumes and networks	`docker volume ls`
	`docker volume create data`
	`docker network ls`
	`docker network create mynet`	

Task	Command	Notes

Docker Compose	`docker compose up -d` *(start in background)*
	`docker compose down` *(stop and remove)*
	`docker compose ps`
	`docker compose logs -f`
	`docker compose build`
	`docker compose exec web sh`
		docker compose (v2, a subcommand) replaces the old docker-compose script. It reads compose.yaml or docker-compose.yml in the current folder.

Keyboard Shortcuts

Action	CMD	PowerShell	Bash	Notes
Auto-complete	`Tab`	`Tab` *(Ctrl+Space shows a menu)*	`Tab` *(press twice to list options)*	
Previous / next command	`↑ / ↓` *(F3 repeats the last one)*	`↑ / ↓`	`↑ / ↓ or Ctrl+P / Ctrl+N`	
Search history	`F7 (list), F8` *(cycle prefix matches)*	`Ctrl+R (backward), Ctrl+S (forward), F8`	`Ctrl+R (backward), Ctrl+S` *(forward)*	
Clear screen	`cls` *(no shortcut)*	`Ctrl+L`	`Ctrl+L`	
Cancel running command	`Ctrl+C`	`Ctrl+C`	`Ctrl+C`	
Clear current line	`Esc`	`Esc`	`Ctrl+U (before cursor), Ctrl+K` *(after cursor)*	In PowerShell, Ctrl+U and Ctrl+K also work with the default key mode.
Start / end of line	`Home / End`	`Home / End` *(also Ctrl+A / Ctrl+E)*	`Ctrl+A / Ctrl+E` *(Home / End in most terminals)*	
Move by word	`Ctrl+← / Ctrl+→`	`Ctrl+← / Ctrl+→`	`Alt+B / Alt+F` *(Ctrl+← / → in many terminals)*	
Delete word before cursor	*(none)*	`Ctrl+Backspace`	`Ctrl+W`	
Delete word after cursor	*(none)*	`Ctrl+Delete`	`Alt+D`	
Undo an edit	*(none)*	`Ctrl+Z`	`Ctrl+_ or Ctrl+X Ctrl+U`	
Paste	`Ctrl+V or right-click`	`Ctrl+V or right-click`	`Ctrl+Shift+V` *(Cmd+V on macOS)*	Copy and paste are handled by the terminal app, not the shell. In Bash Ctrl+C cancels and Ctrl+V inserts a literal key.
Copy	`Select, then Enter or Ctrl+C`	`Select, then Enter or Ctrl+C`	`Ctrl+Shift+C` *(Cmd+C on macOS)*	

Action	CMD	PowerShell	Bash	Notes
Pause / resume output	`Ctrl+S / Ctrl+Q`	`Ctrl+S / Ctrl+Q`	`Ctrl+S / Ctrl+Q`	If the terminal seems frozen, you probably hit Ctrl+S. Press Ctrl+Q.
Suspend foreground job	*(none)*	*(none)*	`Ctrl+Z` *(resume with fg or bg)*	
Exit shell	`exit`	`exit` *(Ctrl+D on an empty line)*	`exit or Ctrl+D`	In CMD, Ctrl+Z is the end-of-file character, not exit.

Notes and Gotchas

Paths

- Windows uses backslashes (C:\Users\Alice). Bash uses forward slashes (/home/alice). PowerShell accepts both.
- Quote any path containing spaces: "C:\Program Files" or '/My Files'.
- Windows file systems are case-insensitive. Linux is case-sensitive: File.txt and file.txt are different files. macOS is case-insensitive by default.

Quoting and escaping

- CMD: escape special characters (& < > | ^) with a caret: echo a ^& b. Variables are %NAME%.
- PowerShell: single quotes are literal, double quotes expand $variables and $(expressions). The escape character is the backtick (`).
- Bash: single quotes are literal, double quotes expand $variables and $(commands). The escape character is the backslash (\).

Wildcards

- * matches any number of characters (*.txt). ? matches exactly one character (file?.txt).
- [abc] and [a-z] match one character from a set (Bash and PowerShell, not CMD).
- In Bash the shell expands wildcards before the command runs. Quote them ("*.txt") when the command should see the pattern itself, as with find and grep.

PowerShell is different

- Commands are Verb-Noun cmdlets (Get-Process, Stop-Service). Aliases such as ls, cat, rm and ps map to cmdlets, but their flags are PowerShell flags: ls -Force, not ls -a.
- Pipelines carry objects, not text. Filter with Where-Object, pick columns with Select-Object, sort with Sort-Object, and format last.
- Windows PowerShell 5.1 ships with Windows. PowerShell 7 (pwsh) is a separate install with && and ||, UTF-8 defaults and Get-Uptime.

macOS, zsh and WSL

- macOS uses zsh as its default shell. Nearly everything in the Bash column works unchanged; profile files are ~/.zshrc instead of ~/.bashrc.
- macOS ships BSD versions of many tools, so some Linux flags differ (sed -i, ls --color, du --max-depth). Commands marked (Linux) do not exist on macOS and vice versa.
- Windows Subsystem for Linux (wsl) gives you a real Linux shell on Windows. Windows drives appear under /mnt/c.

Setting up Windows for development

- Turn on Developer Mode (Settings > System > For developers). It lets you create symlinks without an admin prompt and, on Windows 11 24H2+, enables the built-in sudo.
- Long paths: node_modules trees often exceed the 260-character limit. Run git config --global core.longpaths true, and enable "Enable Win32 long paths" in Group Policy or set LongPathsEnabled=1 under HKLM\SYSTEM\CurrentControlSet\Control\FileSystem (🔒 admin).
- winget installs per user by default; add --scope machine (🔒 admin) to install for everyone.
- Most installers add themselves to the PATH, but only new terminal windows see it. If a command is "not recognized" right after installing, open a new window first.

Git on Windows

- Git for Windows installs Git Bash, a Bash shell where the whole Bash column of this sheet works. Git itself works identically in CMD and PowerShell.
- Line endings: set git config --global core.autocrlf true on Windows (or add a .gitattributes with * text=auto to the repo) so files do not flip between CRLF and LF.
- Credentials: Git for Windows includes Git Credential Manager, which opens a browser sign-in on the first push and remembers it. On macOS the keychain does the same; on Linux use git config --global credential.helper store or the GitHub CLI (gh auth login).
- Lots of "modified" files right after a clone with no real changes usually means a line-ending or file-mode mismatch: git config core.fileMode false and check core.autocrlf.

Command Line Interface Cheat Sheet · © 2026 Fulllion Creative Works · MIT License · also available as DOCX and PDF in the repository.